

PROMPT TÉCNICO PARA ANTIGRAVITY - MÓDULO 4: AGENTES DE IA UNIFICADOS

markdown

CONTEXTO DEL MÓDULO 4

Estás construyendo un sistema de AGENTES DE IA UNIFICADOS que atienden reservas por:

1. WhatsApp (vía Chatwoot + Twilio)
2. Llamadas telefónicas (vía Vapi.ai + Twilio Voice)
3. Chat web (vía Chatwoot)

Todos los canales están unificados en un solo dashboard y crean reservas en Cal.com.

STACK TECNOLÓGICO:

- WhatsApp: Chatwoot + Twilio WhatsApp API
- Voz: Vapi.ai + Twilio Voice + ElevenLabs
- Chat Web: Chatwoot (widget embebible)
- LLM: OpenAI GPT-4o (con Function Calling)
- Backend: Next.js API Routes + Node.js
- Base de Datos: PostgreSQL + Redis (caché)
- Real-time: Socket.io
- Cal.com: API v2 para crear reservas
- LangChain o Vercel AI SDK (orquestración)
- Zod (validación)

ARQUITECTURA GENERAL

****FLUJO DE DATOS:****

WhatsApp → Twilio → Chatwoot → Webhook → Tu Backend → GPT-4o → Cal.com

Voz → Twilio → Vapi.ai → GPT-4o → Cal.com

Chat Web → Chatwoot → Webhook → Tu Backend → GPT-4o → [Cal.com](https://cal.com)

****COMPONENTES PRINCIPALES:****

1. ****Chatwoot**** (Self-hosted o Cloud):

- Centraliza WhatsApp y chat web
- Proporciona interfaz para que humanos tomen control

- Webhooks para enviar mensajes a tu backend
- API para enviar respuestas

2. **Vapi.ai**:

- Maneja llamadas telefónicas entrantes
- Speech-to-text (STT) en tiempo real
- Text-to-speech (TTS) con ElevenLabs
- Function calling para consultar disponibilidad y crear reservas

3. **Tu Backend**:

- Recibe webhooks de Chatwoot
- Procesa mensajes con GPT-4o
- Consulta disponibilidad en Cal.com
- Crea reservas vía Cal.com API
- Envía respuestas a Chatwoot

4. **Dashboard**:

- Muestra conversaciones en tiempo real
- Permite transferir bot ↔ humano
- Estadísticas de agentes IA

SECCIÓN 1: CONFIGURACIÓN DE CHATWOOT

REQUISITOS FUNCIONALES:

INSTALACIÓN Y SETUP:

Opción A: Chatwoot Cloud (SaaS)

- Crear cuenta en chatwoot.com
- Crear inbox de WhatsApp
- Configurar Twilio WhatsApp API
- Obtener API keys

Opción B: Chatwoot Self-hosted

- Deploy en servidor propio (Docker)
- Configurar base de datos PostgreSQL
- Configurar Redis
- Configurar Twilio WhatsApp API

CONFIGURACIÓN DE WHATSAPP:

1. **Twilio Setup:**

- Crear cuenta en Twilio
- Activar WhatsApp Sandbox (testing) o WhatsApp Business API (producción)
- Obtener Account SID y Auth Token
- Configurar webhook de mensajes entrantes

2. **Chatwoot WhatsApp Inbox:**

- Crear nuevo inbox → WhatsApp
- Seleccionar "Twilio" como provider
- Ingresar credenciales de Twilio
- Configurar número de WhatsApp del restaurante
- Probar envío/recepción

3. **Webhooks:**

- Configurar webhook en Chatwoot para:
 - * Mensajes entrantes → Tu backend
 - * Estado de conversación → Tu backend
 - * Transferencia a humano → Tu backend
- Endpoint: `https://tudominio.com/api/webhooks/chatwoot`

CONFIGURACIÓN DE CHAT WEB:

1. **Widget de Chat:**

- Chatwoot genera código JavaScript
- Restaurante pega código en su página web
- Widget aparece en esquina inferior derecha
- Personalizable (colores, posición, mensaje de bienvenida)

2. **Canales adicionales:**

- Facebook Messenger (opcional)
- Instagram DM (opcional)
- Email (opcional)
- Telegram (opcional)

REQUISITOS TÉCNICOS:

Chatwoot API Integration:

```
``typescript
```

```
// lib/chatwoot.ts
```

```
import axios from 'axios';
```

```
const CHATWOOT_API_URL = process.env.CHATWOOT_API_URL;
```

```
const CHATWOOT_TOKEN = process.env.CHATWOOT_TOKEN;
```

```

export async function sendMessageToChatwoot(
  inboxId: string,
  conversationId: string,
  message: string,
  messageType: 'outgoing' | 'template' = 'outgoing'
) {
  const response = await axios.post(
    `${CHATWOOT_API_URL}/api/v1/accounts/1/conversations/${conversationId}/messages`,
    {
      content: message,
      message_type: messageType,
    },
    {
      headers: {
        'api_access_token': CHATWOOT_TOKEN,
        'Content-Type': 'application/json',
      },
    }
  );

  return response.data;
}

```

```

export async function getConversationDetails(conversationId: string) {
  const response = await axios.get(
    `${CHATWOOT_API_URL}/api/v1/accounts/1/conversations/${conversationId}`,
    {
      headers: {
        'api_access_token': CHATWOOT_TOKEN,
      },
    }
  );

  return response.data;
}

```

```

export async function assignConversationToBot(conversationId: string) {
  await axios.post(
    `${CHATWOOT_API_URL}/api/v1/accounts/1/conversations/${conversationId}/toggleStatus`,
    {

```

```

        status: 'open',
    },
    {
        headers: {
            'api_access_token': CHATWOOT_TOKEN,
        }
    }
);

// Agregar etiqueta "bot-attending"
await axios.post(
`${CHATWOOT_API_URL}/api/v1/accounts/1/conversations/${conversationId}/labels`,
{
    labels: ['bot-attending'],
},
{
    headers: {
        'api_access_token': CHATWOOT_TOKEN,
    }
}
);
}

export async function assignConversationToHuman(conversationId: string, userId:
string) {
    await axios.post(
`${CHATWOOT_API_URL}/api/v1/accounts/1/conversations/${conversationId}/toggle
Assign`,
{
    assignee_id: userId,
},
{
    headers: {
        'api_access_token': CHATWOOT_TOKEN,
    }
}
);

// Remover etiqueta "bot-attending"
await axios.post(

```

```

`${CHATWOOT_API_URL}/api/v1/accounts/1/conversations/${conversationId}/labels
`,
{
  labels: ['human-attending'],
},
{
  headers: {
    'api_access_token': CHATWOOT_TOKEN,
  }
}
);
}

```

Webhook Handler:

typescript

```

// pages/api/webhooks/chatwoot.ts
import { NextApiRequest, NextApiResponse } from 'next';
import { handleIncomingMessage } from
'@/lib/ai-handlers/whatsapp';

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed'
  });
}

  const event = req.body;

  // Verificar que es un mensaje nuevo
  if (event.event === 'message_created' &&
event.message.message_type === 'incoming') {
    const { conversation, message } = event;

    // Procesar mensaje con IA
    await handleIncomingMessage({
      conversationId: conversation.id,
      inboxId: conversation.inbox_id,
      contactId: conversation.meta.sender.id,
      contactName: conversation.meta.sender.name,
      contactPhone: conversation.meta.sender.phone_number,
      messageContent: message.content,

```

```

        channel: 'whatsapp',
    });
}

// Evento: conversación transferida a humano
if (event.event === 'conversation_status_changed') {
    const { conversation } = event;

    if (conversation.assignee_id && conversation.assignee_id
    !== null) {
        // Transferido a humano
        await handleTransferToHuman(conversation.id,
        conversation.assignee_id);
    }
}

res.status(200).json({ received: true });
}

```

UI/UX:

- Chatwoot proporciona UI lista para el dashboard
- Personalizar con colores del restaurante
- Configurar respuestas automáticas de bienvenida
- Configurar horarios de atención (fuera de horario → mensaje automático)

SECCIÓN 2: CONFIGURACIÓN DE VAPI (LLAMADAS DE VOZ)

REQUISITOS FUNCIONALES:

SETUP DE VAPI:

1. Crear cuenta en Vapi.ai:
 - Registrarse en vapi.ai
 - Obtener API key
 - Configurar webhook
2. Configurar Twilio Voice:
 - Comprar número telefónico en Twilio
 - Activar Voice API
 - Configurar webhook de llamadas entrantes
 - Conectar con Vapi
3. Configurar Assistant en Vapi:

- Crear nuevo assistant
- Configurar modelo: GPT-4o
- Configurar voz: ElevenLabs (seleccionar voz en español)
- Configurar system prompt
- Configurar functions (tools)

FUNCTIONS/TOOLS PARA EL ASISTENTE DE VOZ:

El asistente necesita estas funciones:

1. `checkAvailability`:
 - Consulta disponibilidad en Cal.com
 - Parámetros: fecha, hora, número de personas
 - Retorna: horarios disponibles
2. `createBooking`:
 - Crea reserva en Cal.com
 - Parámetros: fecha, hora, personas, nombre, teléfono, email
 - Retorna: confirmation code
3. `getRestaurantInfo`:
 - Información del restaurante (horarios, dirección, menú)
 - Sin parámetros
 - Retorna: información general
4. `transferToHuman`:
 - Transfiere llamada a humano
 - Parámetros: reason
 - Retorna: success

REQUISITOS TÉCNICOS:

Vapi Configuration:

```
typescript
// lib/vapi.ts
import Vapi from '@vapi-ai/web';

const vapi = new Vapi(process.env.NEXT_PUBLIC_VAPI_API_KEY!);

export interface AssistantConfig {
  model: {
    provider: 'openai';
    model: 'gpt-4o';
  };
  voice: {
    provider: 'elevenlabs';
    voiceId: string; // ej: "21m00Tcm4TlvDq8ikWAM" (Rachel)
  };
};
```

```
    firstMessage: string;
    systemPrompt: string;
    functions: FunctionConfig[];
  }

export const restaurantAssistantConfig: AssistantConfig = {
  model: {
    provider: 'openai',
    model: 'gpt-4o',
  },
  voice: {
    provider: 'elevenlabs',
    voiceId: '21m00Tcm4TlvDq8ikWAM', // Rachel - español
    neutro
  },
  firstMessage: '¡Hola! Bienvenido a [Nombre del Restaurante]. Soy tu asistente virtual. ¿En qué puedo ayudarte hoy?',
  systemPrompt: `Eres un asistente virtual amable y profesional para un restaurante. Tu objetivo es ayudar a los clientes a hacer reservas.
```

INSTRUCCIONES:

1. Saluda de manera cálida y profesional
2. Pregunta para cuántas personas y qué fecha/hora desean
3. Usa la función `checkAvailability` para verificar disponibilidad
4. Si hay disponibilidad, ofrece los horarios disponibles
5. Cuando el cliente elija, usa `createBooking` para hacer la reserva
6. Confirma la reserva con el código de confirmación
7. Ofrece ayuda adicional si es necesario
8. Si no puedes ayudar, ofrece transferir a un humano

TONO:

- Amable, profesional, pero cercano
- Habla en español neutro
- Sé conciso pero claro
- Repite información importante (fecha, hora, número de personas)

REGLAS:

- Nunca inventes disponibilidad, siempre consulta con `checkAvailability`
- Confirma siempre los datos antes de crear la reserva

- Si el cliente pide algo fuera de lo normal, ofrece transferir a humano

- Maneja objeciones con empatía`,

```
functions: [  
  {  
    name: 'checkAvailability',  
    description: 'Verifica disponibilidad de mesas para una  
fecha, hora y número de personas',  
    parameters: {  
      type: 'object',  
      properties: {  
        date: {  
          type: 'string',  
          description: 'Fecha en formato YYYY-MM-DD',  
        },  
        time: {  
          type: 'string',  
          description: 'Hora en formato HH:mm',  
        },  
        partySize: {  
          type: 'number',  
          description: 'Número de personas',  
        },  
      },  
      required: ['date', 'partySize'],  
    },  
  },  
  {  
    name: 'createBooking',  
    description: 'Crea una reserva en el sistema',  
    parameters: {  
      type: 'object',  
      properties: {  
        date: {  
          type: 'string',  
          description: 'Fecha en formato YYYY-MM-DD',  
        },  
        time: {  
          type: 'string',  
          description: 'Hora en formato HH:mm',  
        },  
        partySize: {  
          type: 'number',  
          description: 'Número de personas',  
        },  
      },  
    },  
  },  
]
```

```

        },
        customerName: {
            type: 'string',
            description: 'Nombre completo del cliente',
        },
        customerPhone: {
            type: 'string',
            description: 'Teléfono del cliente',
        },
        customerEmail: {
            type: 'string',
            description: 'Email del cliente',
        },
    },
    required: ['date', 'time', 'partySize',
'customerName', 'customerPhone'],
},
{
    name: 'getRestaurantInfo',
    description: 'Obtiene información del restaurante
(horarios, dirección, menú)',
    parameters: {
        type: 'object',
        properties: {},
    },
},
{
    name: 'transferToHuman',
    description: 'Transfiere la llamada a un agente humano',
    parameters: {
        type: 'object',
        properties: {
            reason: {
                type: 'string',
                description: 'Razón de la transferencia',
            },
        },
    },
    required: ['reason'],
},
],
};

```

```

// Iniciar llamada
export async function startCall(assistantId: string) {
  vapi.start(assistantId, {
    onCallStart: () => {
      console.log('Llamada iniciada');
    },
    onCallEnd: () => {
      console.log('Llamada finalizada');
    },
    onMessage: (message) => {
      console.log('Mensaje:', message);
    },
    onError: (error) => {
      console.error('Error:', error);
    },
  });
}

// Funciones que Vapi llamará
export const functionHandlers = {
  checkAvailability: async (parameters: any) => {
    const response = await fetch('/api/calcom/availability', {
      method: 'POST',
      body: JSON.stringify(parameters),
    });
    return response.json();
  },

  createBooking: async (parameters: any) => {
    const response = await fetch('/api/calcom/bookings', {
      method: 'POST',
      body: JSON.stringify(parameters),
    });
    return response.json();
  },

  getRestaurantInfo: async () => {
    return {
      name: 'Mi Gran Parrilla',
      address: 'Calle 123 #45-67, Bogotá',
      hours: 'Lun-Dom: 12:00 PM - 11:00 PM',
      phone: '+57 300 123 4567',
    };
  },
};

```

```

transferToHuman: async (parameters: any) => {
  // Lógica para transferir a humano
  return { success: true };
},
};

```

Webhook de Vapi:

typescript

// pages/api/webhooks/vapi.ts

```
import { NextApiRequest, NextApiResponse } from 'next';
```

```

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed'
  });
}

```

```
const event = req.body;
```

```

// Evento: Llamada iniciada
if (event.type === 'call.started') {
  await handleCallStarted(event.call);
}

```

```

// Evento: Llamada finalizada
if (event.type === 'call.ended') {
  await handleCallEnded(event.call);
}

```

```

// Evento: Function call solicitada por el asistente
if (event.type === 'function-call') {
  const { functionName, parameters, callId } = event;

```

```

  const result = await
functionHandlers[functionName](parameters);

```

```

  // Enviar resultado de vuelta a Vapi
  await
fetch(`https://api.vapi.ai/call/${callId}/function-response`,
{
  method: 'POST',

```

```

    headers: {
      'Authorization': `Bearer ${process.env.VAPI_API_KEY}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ result }),
  });
}

```

```

// Evento: Transferencia a humano solicitada
if (event.type === 'transfer-requested') {
  await handleTransferToHuman(event.call, event.reason);
}

```

```

  res.status(200).json({ received: true });
}

```

Twilio Voice Configuration:

```

typescript
// pages/api/twilio/voice.ts
import { NextApiRequest, NextApiResponse } from 'next';
import twilio from 'twilio';

const VoiceResponse = twilio.twiml.VoiceResponse;

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const twiml = new VoiceResponse();

  // Conectar con Vapi
  twiml.say('Conectando con el asistente virtual...');

  // Redirigir a Vapi
  twiml.redirect({
    method: 'POST',
    url:
`https://api.vapi.ai/call/${process.env.VAPI_ASSISTANT_ID}`,
  });

  res.setHeader('Content-Type', 'text/xml');
  res.send(twiml.toString());
}

```

UI/UX:

- Indicador visual de "Llamada en curso" en el dashboard
 - Transcripción en tiempo real de la llamada
 - Botón "Tomar control" (transferir a humano)
 - Visualización de la conversación (qué dijo el cliente, qué respondió el bot)
 - Métricas: duración de llamada, estado, reserva creada (sí/no)
-

SECCIÓN 3: CEREBRO DE IA (GPT-4O)

REQUISITOS FUNCIONALES:

SYSTEM PROMPTS:

Crear prompts específicos para cada canal:

1. WhatsApp/Chat:
 - Más conciso (mensajes cortos)
 - Emojis apropiados
 - Formato amigable
 - Soporta envío de imágenes/links
2. Voz (Vapi):
 - Más conversacional
 - Frases más largas
 - Sin emojis (no se pueden decir)
 - Confirmaciones verbales
3. Chat Web:
 - Similar a WhatsApp
 - Puede incluir botones/quick replies
 - Soporta rich media

CONTEXT MANAGEMENT:

- Mantener contexto de la conversación (últimos 10 mensajes)
- Recordar información del cliente (nombre, preferencias)
- Acceder a historial de reservas del cliente
- Persistir estado entre sesiones (Redis)

FUNCTION CALLING:

El LLM debe poder llamar estas funciones:

1. checkAvailability(restaurantId, branchId, date, partySize)
2. createBooking(restaurantId, branchId, date, time, partySize, customer)
3. getCustomerHistory(customerPhone)
4. getRestaurantInfo(restaurantId)
5. sendConfirmationMessage(customerPhone, bookingDetails)

6. transferToHuman(reason)

REQUISITOS TÉCNICOS:

AI Handler:

```
typescript
// lib/ai-handlers/whatsapp.ts
import OpenAI from 'openai';
import { sendMessageToChatwoot } from '@lib/chatwoot';
import { checkAvailability, createBooking } from
'@lib/calcom';

const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});

const SYSTEM_PROMPT = `Eres un asistente virtual para reservas
de restaurantes. Tu objetivo es ayudar a los clientes a hacer
reservas de manera rápida y amable.`
```

INSTRUCCIONES:

1. Saluda de manera cálida
2. Pregunta: número de personas, fecha y hora preferida
3. Verifica disponibilidad antes de confirmar
4. Confirma todos los datos antes de crear la reserva
5. Proporciona código de confirmación
6. Ofrece ayuda adicional

TONO:

- Amable, profesional, cercano
- Usa emojis moderadamente 😊
- Sé conciso pero claro
- Español neutro

REGLAS:

- Nunca inventes disponibilidad
- Siempre confirma antes de crear reserva
- Si no entiendes, pide clarificación
- Si no puedes ayudar, ofrece transferir a humano`;

```
export async function handleIncomingMessage(data: {
  conversationId: string;
  inboxId: string;
  contactId: string;
```

```

    contactName: string;
    contactPhone: string;
    messageContent: string;
    channel: 'whatsapp' | 'web';
  }) {
    // Obtener historial de conversación (últimos 10 mensajes)
    const conversationHistory = await
getConversationHistory(data.conversationId);

    // Obtener información del cliente
    const customer = await getOrCreateCustomer({
      name: data.contactName,
      phone: data.contactPhone,
    });

    // Obtener historial de reservas del cliente
    const bookingHistory = await
getCustomerBookingHistory(customer.id);

    // Construir mensajes para GPT
    const messages: OpenAI.Chat.ChatCompletionMessageParam[] = [
      { role: 'system', content: SYSTEM_PROMPT },
      ...conversationHistory,
      { role: 'user', content: data.messageContent },
    ];

    // Agregar contexto del cliente
    if (bookingHistory.length > 0) {
      messages.push({
        role: 'system',
        content: `Este cliente ha visitado el restaurante
${bookingHistory.length} veces. Última visita:
${bookingHistory[0].date}.`,
      });
    }

    // Llamar a GPT-4o con function calling
    const response = await openai.chat.completions.create({
      model: 'gpt-4o',
      messages,
      tools: [
        {
          type: 'function',
          function: {

```

```
        name: 'checkAvailability',
        description: 'Verifica disponibilidad de mesas',
        parameters: {
            type: 'object',
            properties: {
                date: { type: 'string', description: 'Fecha
YYYY-MM-DD' },
                time: { type: 'string', description: 'Hora
HH:mm' },
                partySize: { type: 'number', description:
'Número de personas' },
            },
            required: ['date', 'partySize'],
        },
    },
    {
        type: 'function',
        function: {
            name: 'createBooking',
            description: 'Crea una reserva',
            parameters: {
                type: 'object',
                properties: {
                    date: { type: 'string' },
                    time: { type: 'string' },
                    partySize: { type: 'number' },
                    customerName: { type: 'string' },
                    customerPhone: { type: 'string' },
                    customerEmail: { type: 'string' },
                    specialRequests: { type: 'string' },
                },
                required: ['date', 'time', 'partySize',
'customerName', 'customerPhone'],
            },
        },
    },
    {
        type: 'function',
        function: {
            name: 'transferToHuman',
            description: 'Transfiere a un agente humano',
            parameters: {
                type: 'object',
```

```

        properties: {
            reason: { type: 'string' },
        },
        required: ['reason'],
    },
},
],
    tool_choice: 'auto',
});

const assistantMessage = response.choices[0].message;

// Si el asistente quiere llamar una función
if (assistantMessage.tool_calls) {
    for (const toolCall of assistantMessage.tool_calls) {
        const functionName = toolCall.function.name;
        const args = JSON.parse(toolCall.function.arguments);

        let result;

        if (functionName === 'checkAvailability') {
            result = await checkAvailability(args);
        } else if (functionName === 'createBooking') {
            result = await createBooking({
                ...args,
                source: 'whatsapp',
            });

            // Enviar confirmación
            await sendConfirmationMessage(data.contactPhone,
result);
        } else if (functionName === 'transferToHuman') {
            await transferToHuman(data.conversationId,
args.reason);
            result = { success: true };
        }

        // Agregar resultado al contexto
        messages.push({
            role: 'tool',
            tool_call_id: toolCall.id,
            content: JSON.stringify(result),
        });
    }
}

```

```

    }

    // Obtener respuesta final del asistente
    const finalResponse = await
openai.chat.completions.create({
    model: 'gpt-4o',
    messages,
});

    const replyContent =
finalResponse.choices[0].message.content;

    // Enviar respuesta a Chatwoot
    await sendMessageToChatwoot(
        data.inboxId,
        data.conversationId,
        replyContent!,
        'outgoing'
    );
} else {
    // Respuesta directa sin function calling
    await sendMessageToChatwoot(
        data.inboxId,
        data.conversationId,
        assistantMessage.content!,
        'outgoing'
    );
}

// Guardar mensajes en historial
await saveConversationHistory(data.conversationId, [
    { role: 'user', content: data.messageContent },
    { role: 'assistant', content: assistantMessage.content },
]);
}

```

Context Management con Redis:

```

typescript
// lib/redis.ts
import Redis from 'ioredis';

const redis = new Redis(process.env.REDIS_URL!);

export async function getConversationHistory(conversationId:
string) {

```

```

    const history = await
redis.get(`conversation:${conversationId}:history`);
    return history ? JSON.parse(history) : [];
}

export async function saveConversationHistory(
  conversationId: string,
  messages: Array<{ role: string; content: string }>
) {
  const key = `conversation:${conversationId}:history`;

  // Obtener historial existente
  const existing = await
getConversationHistory(conversationId);

  // Agregar nuevos mensajes
  const updated = [...existing, ...messages].slice(-20); //
Mantener últimos 20

  // Guardar con TTL de 24 horas
  await redis.setex(key, 86400, JSON.stringify(updated));
}

export async function clearConversationHistory(conversationId:
string) {
  await redis.del(`conversation:${conversationId}:history`);
}

```

UI/UX:

- Indicador de "IA escribiendo..." en el chat
- Transcripción en tiempo real
- Visualización de funciones llamadas (checkAvailability, createBooking)
- Badge "Bot" vs "Humano" en mensajes
- Opción "¿Fue útil esta respuesta?" (feedback)

SECCIÓN 4: INTEGRACIÓN CON CAL.COM

REQUISITOS FUNCIONALES:

FUNCIONES NECESARIAS:

1. checkAvailability:
 - Consulta Cal.com API v2

- Verifica disponibilidad para fecha/hora/personas
- Retorna slots disponibles
- 2. createBooking:
 - Crea reserva en Cal.com
 - Agrega metadata (source: whatsapp/voice/web)
 - Retorna confirmation code
- 3. getEventTypes:
 - Obtiene tipos de evento configurados
 - Para mostrar opciones al cliente
- 4. cancelBooking:
 - Cancela reserva existente
 - Procesa reembolso si aplica

REQUISITOS TÉCNICOS:

```
typescript
// lib/calcom.ts
import axios from 'axios';

const CALCOM_API_URL = process.env.CALCOM_API_URL ||
'https://app.cal.com';
const CALCOM_API_KEY = process.env.CALCOM_API_KEY!;

const headers = {
  'Authorization': `Bearer ${CALCOM_API_KEY}`,
  'Content-Type': 'application/json',
};

export async function checkAvailability({
  eventTypeid,
  date,
  partySize,
}): {
  eventTypeid: number;
  date: string;
  partySize: number;
}) {
  const startDate = `${date}T00:00:00Z`;
  const endDate = `${date}T23:59:59Z`;

  const response = await axios.get(
    `${CALCOM_API_URL}/api/v2/slots`,
    {
      headers,
    }
  );
}
```

```

        params: {
            eventTypeId,
            startTime: startDate,
            endTime: endDate,
        },
    }
);

// Filtrar slots por capacidad
const availableSlots = response.data.slots.filter(
    (slot: any) => slot.attendees < partySize
);

return {
    available: availableSlots.length > 0,
    slots: availableSlots.map((slot: any) => slot.time),
};
}

export async function createBooking({
    eventTypeId,
    date,
    time,
    partySize,
    customerName,
    customerPhone,
    customerEmail,
    specialRequests,
    source,
}: {
    eventTypeId: number;
    date: string;
    time: string;
    partySize: number;
    customerName: string;
    customerPhone: string;
    customerEmail?: string;
    specialRequests?: string;
    source: 'whatsapp' | 'voice' | 'web';
}) {
    const start = `${date}T${time}:00`;

    const response = await axios.post(
        `${CALCOM_API_URL}/api/v2/bookings`,

```

```

    {
      eventTypeid,
      start,
      responses: {
        name: customerName,
        email: customerEmail ||
`${customerPhone}@reservations.local`,
        phone: customerPhone,
        partySize,
        notes: specialRequests,
      },
      metadata: {
        bookingSource: source,
        customerPhone,
      },
      language: 'es',
      timeZone: 'America/Bogota',
    },
    { headers }
  );

```

```

  return {
    bookingId: response.data.id,
    confirmationCode: response.data.uid,
    status: response.data.status,
  };
}

```

```

export async function getEventTypes(restaurantId: string) {
  const response = await axios.get(
    `${CALCOM_API_URL}/api/v2/event-types`,
    {
      headers,
      params: { teamId: restaurantId },
    }
  );

  return response.data.eventTypes;
}

```

```

export async function cancelBooking(bookingId: string, reason:
string) {
  const response = await axios.delete(
    `${CALCOM_API_URL}/api/v2/bookings/${bookingId}`,

```

```
{
  headers,
  data: { reason },
}
);

return response.data;
}
```

SECCIÓN 5: DASHBOARD EN TIEMPO REAL

REQUISITOS FUNCIONALES:

VISTA DE CONVERSACIONES:

- Lista de conversaciones activas
- Indicador de canal (WhatsApp, Voz, Web)
- Indicador de estado (Bot atendiendo / Humano atendiendo)
- Preview del último mensaje
- Timestamp
- Badge de no leídos

DETALLE DE CONVERSACIÓN:

- Transcripción completa
- Indicador de quién habla (Bot/Cliente/Humano)
- Timestamp de cada mensaje
- Funciones llamadas por el bot (checkAvailability, createBooking)
- Resultado de funciones
- Reserva creada (si aplica)

CONTROLES:

- Botón "Tomar control" (transferir bot → humano)
- Botón "Transferir a bot" (humano → bot)
- Input para escribir mensaje manual
- Respuestas rápidas predefinidas
- Botón "Marcar como resuelta"

ESTADÍSTICAS:

- Conversaciones totales (hoy/semana/mes)
- Por canal (WhatsApp, Voz, Web)
- Tasa de resolución del bot (%)
- Tiempo promedio de respuesta
- Reservas generadas por IA

- Transferencias a humano (%)

REQUISITOS TÉCNICOS:

```
typescript
// components/ConversationsDashboard.tsx
import { useEffect, useState } from 'react';
import { io, Socket } from 'socket.io-client';

export default function ConversationsDashboard() {
  const [conversations, setConversations] =
    useState<Conversation[]>([]);
  const [selectedConversation, setSelectedConversation] =
    useState<string | null>(null);
  const [socket, setSocket] = useState<Socket | null>(null);

  useEffect(() => {
    // Conectar a WebSocket
    const newSocket = io(process.env.NEXT_PUBLIC_WS_URL!);
    setSocket(newSocket);

    // Suscribirse a nuevas conversaciones
    newSocket.on('new-conversation', (conversation:
    Conversation) => {
      setConversations(prev => [conversation, ...prev]);
    });

    // Suscribirse a nuevos mensajes
    newSocket.on('new-message', (message: Message) => {
      setConversations(prev =>
        prev.map(conv =>
          conv.id === message.conversationId
            ? { ...conv, lastMessage: message, updatedAt:
message.createdAt }
            : conv
        )
      );
    });

    // Suscribirse a transferencias
    newSocket.on('conversation-transferred', (data: {
conversationId: string, to: 'bot' | 'human' }) => {
      setConversations(prev =>
        prev.map(conv =>
```

```

        conv.id === data.conversationId
        ? { ...conv, assignedTo: data.to }
        : conv
    )
  );
});

return () => {
  newSocket.disconnect();
};
}, []);

return (
  <div className="flex h-screen">
    {/* Lista de conversaciones */}
    <div className="w-1/3 border-r">
      <ConversationsList
        conversations={conversations}
        selectedId={selectedConversation}
        onSelect={setSelectedConversation}
      />
    </div>

    {/* Detalle de conversación */}
    <div className="flex-1">
      {selectedConversation ? (
        <ConversationDetail
          conversationId={selectedConversation}
          socket={socket}
        />
      ) : (
        <div className="flex items-center justify-center h-full text-gray-500">
          Selecciona una conversación
        </div>
      )}
    </div>
  </div>
);
}

```

UI/UX:

- Diseño tipo WhatsApp Web / Intercom
- Notificaciones toast cuando llega mensaje nuevo

- Sonido de notificación (opcional)
 - Badges de no leídos
 - Búsqueda de conversaciones
 - Filtros por canal y estado
 - Responsive (móvil: una pantalla a la vez)
-

SECCIÓN 6: ESTADÍSTICAS Y ANALYTICS

REQUISITOS FUNCIONALES:

MÉTRICAS PRINCIPALES:

1. Volumen:
 - Total de conversaciones (día/semana/mes)
 - Por canal (WhatsApp, Voz, Web)
 - Conversaciones activas ahora
2. Rendimiento del Bot:
 - Tasa de resolución automática (%)
 - Conversaciones transferidas a humano (%)
 - Tiempo promedio de respuesta del bot
 - Satisfacción del cliente (rating)
3. Reservas:
 - Reservas generadas por IA
 - Por canal (WhatsApp vs Voz vs Web)
 - Tasa de conversión (conversación → reserva)
 - Ingresos estimados
4. Llamadas (Voz):
 - Total de llamadas
 - Duración promedio
 - Tasa de abandono
 - Calidad de audio (MOS score)
5. WhatsApp:
 - Mensajes enviados/recibidos
 - Tiempo de respuesta promedio
 - Tasa de apertura

GRÁFICOS:

- Conversaciones por día (línea)
- Distribución por canal (donut)
- Reservas generadas por canal (barras)
- Tasa de resolución del bot (gauge)
- Duración de llamadas (histograma)

REQUISITOS TÉCNICOS:

```
typescript
// pages/api/analytics/conversations.ts
export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse
) {
  const { startDate, endDate } = req.query;

  const stats = await prisma.$queryRaw`
    SELECT
      COUNT(*) as totalConversations,
      COUNT(CASE WHEN assignedTo = 'bot' THEN 1 END) as
botHandled,
      COUNT(CASE WHEN assignedTo = 'human' THEN 1 END) as
humanHandled,
      AVG(EXTRACT(EPOCH FROM (lastMessageAt - createdAt))) as
avgResponseTime,
      COUNT(CASE WHEN status = 'resolved' THEN 1 END) as
resolvedCount
    FROM conversations
    WHERE "restaurantId" = ${req.query.restaurantId}
      AND createdAt >= ${startDate}
      AND createdAt <= ${endDate}
  `;

  const bookingsBySource = await prisma.booking.groupBy({
    by: ['source'],
    _count: true,
    where: {
      restaurantId: req.query.restaurantId,
      createdAt: {
        gte: new Date(startDate as string),
        lte: new Date(endDate as string),
      },
    },
  });

  res.json({
    stats,
    bookingsBySource,
  });
}
```

UI/UX:

- Dashboard con cards de KPIs
 - Gráficos interactivos (Recharts)
 - Filtros de fecha
 - Exportación a PDF/Excel
 - Actualización en tiempo real
-

RESUMEN DE ENTREGABLES ESPERADOS

Después de implementar este módulo, debes tener:

✓ Chatwoot:

- Configurado con WhatsApp (Twilio)
- Configurado con chat web
- Webhooks funcionando
- API integration completa

✓ Vapi.ai:

- Assistant configurado con GPT-4o
- Funciones de checkAvailability y createBooking
- Integración con Twilio Voice
- Webhooks funcionando

✓ IA/Backend:

- GPT-4o con function calling
- Context management con Redis
- Handlers para WhatsApp y voz
- Integración con Cal.com

✓ Dashboard:

- Lista de conversaciones en tiempo real
- Detalle de conversación
- Controles de transferencia (bot ↔ humano)
- Estadísticas y analytics

✓ Cal.com Integration:

- checkAvailability
- createBooking
- cancelBooking

- Metadata de origen (whatsapp/voice/web)

✓ Real-time:

- WebSocket para actualizaciones
- Notificaciones push
- Transcripción en vivo

✓ Analytics:

- Métricas de conversaciones
 - Métricas de reservas
 - Métricas de rendimiento del bot
 - Gráficos y exportación
-

INSTRUCCIONES PARA ANTIGRAVITY

1. Configura Chatwoot (self-hosted o cloud)
2. Configura Twilio (WhatsApp + Voice)
3. Configura Vapi.ai (assistant, functions, webhook)
4. Implementa backend (Next.js API routes para webhooks)
5. Implementa IA handlers (GPT-4o con function calling)
6. Integra con Cal.com (availability, bookings)
7. Construye dashboard (conversaciones, real-time)
8. Agrega analytics (métricas, gráficos)
9. Testing (WhatsApp, llamadas, chat web)
10. Optimización (performance, errores)

Prioridad:

1. Chatwoot + WhatsApp básico
2. GPT-4o integration
3. Cal.com integration
4. Vapi.ai (voz)
5. Dashboard en tiempo real
6. Analytics

Nota: Cada canal (WhatsApp, voz, web) debe funcionar independientemente pero compartir el mismo cerebro de IA y crear reservas en el mismo calendario de Cal.com.